

Fordítóprogramok készítése
BMEVIIIML10 – Önálló laboratórium 1
Beszámoló

Bodor András

2025. május 22.

Tartalomjegyzék

1. A feladat leírása	2
1.1. Feladatkiírás	2
1.2. Motiváció	2
1.3. Általános elvárások	3
2. A nyelv ismertetése	5
2.1. A nyelv áttekintő leírása	5
2.2. A nyelv családja és ismerősei	6
2.3. A nyelv felépítése	7
2.3.1. Literálisok	7
2.3.2. Változók	7
2.3.3. Függvények	8
2.3.4. Blokkok	8
2.3.5. Lustaság	10
3. Megvalósítás	11
3.1. A lexikális feldolgozás	11
3.2. Az értelmező	11
3.2.1. Hibaüzenetek	12
3.2.2. Infix operátorok	12
3.2.3. Függvényhívások	13
3.3. LLVM IR generálása	14
3.3.1. Változók	14
3.3.2. Lusta paraméterek	15
3.3.3. Closure blokkok	16
Rövidítésjegyzék	17
Szójegyzék	19
Irodalomjegyzék	20

1. fejezet

A feladat leírása

1.1. Feladatkiírás

A szoftverfejlesztést nagyban megkönnyíthetik az olyan eszközök, amelyek leve-
szik a programozó válláról a mechanikusan végrehajtandó feladatokat (pl. JPA en-
titások készítése, REST interfész generálása, DTO objektumok definiálása, stb.).
Sokszor egy egyszerű, a problémakörhöz közelebb álló szakterület-specifikus
programnyelv (domain-specific language) hatékonyabb munkát biztosíthat, mint
az általános programnyelvek segítségével elvégezni a feladatot.

A téma során a hallgató feladata egy egyszerű programnyelv definiálása, majd
ehhez egy fordítóprogram írása. A fordítóprogram készítése során front-end (pl.
Xtext, ANTLR4) és back-end (pl. LLVM) eszközök is használhatók.

1.2. Motiváció

A feladat célja, egy olyan nyelv elkészítése, amelyiket lehetséges lesz a későbbiek-
ben továbbfejleszteni egy általános C és C++ kódbázisokban mintázatokat kereső
eszközzé. Azon keresztül pedig, hogy kódrészleteket lehet keresni egy adott kód-
bázisban, különböző módon lehetséges reagálni. Az egyik felhasználási lehetőség
lehet házi feladatok ellenőrzésénél egyes tárgyak elvárásainak teljesülését vizs-
gálni. De például ha dokumentáció nem a kódban található, akkor annak a megléte
a megfelelő külső fájlban is ellenőrizhetővé válik, a kíván művelet implementálá-
sával.

Felmerülhet a kérdés, hogy miben más ez/miben különül el, a már meglévő
és akár nyílt forráskódú, akár kommersz statikus analízist végrehajtó eszközök
széles tárházától. Ezek az eszközök nagyszerűen tudnak nyelv-specifikus hibás
idiómákat (pl. virtuális destruktorki hiánya egy őssztályban C++-ban), és általános

code–smelleket (pl. túlságosan nagy osztály/függvény) megkeresni és jelezni a fejlesztők felé. Sok esetben a kedvenc fejlesztőkörnyezetünkben léteznek hozzájuk megfelelő bővítmények, amelyek a hibás/hibásnak vélt kódrészleteket aláhúzzák, így kódolás közben, valós időben is tájékoztatnak a nem feltétlen szerencsés kódrészletekről.

A végső cél projekt az előbb leírt gondolatnak egy általánosított változatát kívánja megvalósítani. Az elérhető statikus analízis eszközök nagy része előre konfigurált mintázatokat keresnek egy-egy kódbázisban, és új felvételéhez a fejlesztőknek azt az eszköznek megfelelő módon bele kell illeszteniük, amit a következő verziószám alatt elérhetővé lehet tenni. Így működik a C és C++ nyelvekhez az LLVM projekt gondozásában készült „clang-tidy”. [LLV] Esetlegesen, ha mégis támogatják saját minták felvételét, akkor sem garantált, hogy tetszőleges kód futtatását lehetővé teszik azon mintázatok megtalálásának hatására.

Itt ki kell emelni, hogy egyes nyelvekhez (pl. Ruby) léteznek ennél jóval inkább dinamikus statikus analízis eszközök (a Ruby nyelvhez konkrétan a RuboCop [Rub], amelyik dinamikusan tud betölteni ellenőrző „cop” (rendőr) osztályokat egy-egy projektben saját célokra), de ilyenek eszközről nem tudok a C és C++ nyelvek ökoszisztémájában.

1.3. Általános elvárások

Az előzőek ismeretében ebben a fejezetben olyan követelményeket/elvárásokat rögzítetek, amelyet a nyelv jövőbeli használatát hivatottak lehetővé tenni a bemutatott feladatra.

Tulajdonság 1. Fordítás nélkül futtatható.

A nyelvnek lehetőség szerint nem fordítotttnak kell lennie, hogy ne kelljen a különböző ellenőrzések fordításával foglalkozni ahhoz, hogy egy-egy projekt le tudjon ellenőrizni valamilyen saját elvárást a kódjában. Ez a tulajdonság hasznossá válik pipeline-okban való használat esetén is, mert nem szükségelteti, hogy egy külön lépést definiáljunk csak az ellenőrzések fordítására. Ennek az elvárást teljesíti egy egyszerű interpreter de egy komplett **Just-In-Time compiler (JIT)** rendszer is. Az alább leírt megvalósítás az utóbbit használja.

Tulajdonság 2. Dinamikusan típusos

A mai napig heves (hit)vita folyik a statikusan és dinamikusan típusos nyelveket támogató táborok között az egyik másiknál jobbnak létéről. Klasszikusan a különböző interpretált ún. „script nyelvek” kategorizált nyelvek utóbbi csoportba tartoznak, pl. Perl, Ruby, Python, Tcl, és a különböző Shell nyelvek (Bash, ZSH, tcsh,

CMD.EXE), míg a fordított nyelvek inkább statikus típusokkal dolgoznak, pl. C, C++, Java, C#, Rust. Ennek alapja a nyelvek alapvető célja: ha rövid „scripteket” kíván valaki írni, akkor általában valami rövid, egyszerű logikát megvalósító egyszer-egyszer magában lefuttatott kódra gondolunk, aminek általában nem elsődleges célja a hosszú távú karbantarthatóság, vagy pusztán a kód kis mérete miatt ez nem számít nehézségnek, így a típusok nem számítanak annyira hasznosnak. Ennek a gondolatmenetnek következményeként, egy vélhetően rövid ellenőrző „script” esetén a dinamikus típusok megkönnyítik a gyors fejlesztést, hogy a megfelelő kódrészleteket meg tudjuk keresni, és valamit tudjunk velük csinálni.

Tulajdonság 3. Erős beágyazott **Domain-Specific Language (DSL)** támogatás.

Mivel a nyelven elsődlegesen különböző kódrészleteket szeretnénk megkeresni, és azokat megtalálva valamilyen reakciót lefuttatni, így két lehetőség adódik: a nyelv szintaktikájába alapról beépíteni a keresés lehetőségét vagy egy olyan általános lehetőséget biztosítani, amin keresztül lehetséges lesz, hogy a nyelven belül kerüljön ez definiálásra. Az első megoldásra példa az AWK nyelv, amelyikben a különböző regexek és a sikeres találatukkor lefutott kód erősen része a nyelv szintaktikájának.

Mivel itt elsődlegesen a C és C++ nyelvek **Abstract Syntax Tree (AST)** alapú leírásában próbálunk keresni, így a második lehetőség kényelmesebb eredményt nyújthat: ezek a nyelvek gyakran és előre várható periodicitással változnak (~3 év-ente), így elég dinamikusságot kell kezelni a kereső szintaktikában.

Tulajdonság 4. Egyszerűen érthető egy C vagy C++ fejlesztőnek.

Elsődlegesen C és C++ programozók azok, akik C vagy C++ kódban szeretnének kódrészleteket megkeresni, így a megvalósított nyelvnek legalább szintaktikában jó lenne hasonlítani ezekre a nyelvekre, hogy ne növeljük meg fölöslegesen az érthetőséget, vagy a kód írásának befektetett energiaigényét. Bár ennek a tulajdonságnak a legegyszerűbb kielégítése C vagy C++ használata lenne, az előző tulajdonságokban leírtak miatt ez nem egy ideális választás.

2. fejezet

A nyelv ismertetése

2.1. A nyelv áttekintő leírása

Az előző tulajdonságok alapján megalkotott nyelv leírása következik. Magas szinten, egy olyan nyelvet definiálok, amelyik funkcionális, erősen immutábilis, dinamikus típusos, és nagyon dinamikus a szintaktikáját tekintve.

Ezeknek a tulajdonságoknak nagy része megfelel és direkt következménye az előző részben definiált elvárásoknak. A funkcionális, immutábilis nyelv választása azonban nem egyértelmű: a klasszikus könnyen futtatható, dinamikus nyelvek nagy többségben egyszerű procedurális paradigma szerint működnek ld. Tcl, shell nyelvek, esetleg utólagosan különféle módon objektum-orientált támogatást kaptak ilyen pl. a Perl. Ezekben a nyelvekben látszik, hogy az egyszerű procedurális paradigma nagyon kevés segítséget biztosít a programozónak a kód megírása közben: nagyrészen imperatív kódot kell, egymást hívó procedurák sorozataként modellezni, magasabb szintű absztrakciók nyelvi támogatottsága nélkül. (ld. C-ben (nem „script” nyelv, de procedurális) objektum-orientált programozáshoz manuálisan kell megvalósítani a virtuális függvénytáblát.)

A megvalósított nyelvnek a funkcionális irányzatot választottam tehát a procedurális és objektum orientált helyett. Az előbbi megközelítés nagyon kevés támogatást ad a programozónak, míg az utóbbi nagy mennyiségű extra kódot igényelhet (osztályok/metódusok definiálása, példányosítások) egy ilyen egyszerű célú kódot támogató eszköz számára, ami elsődlegesen nagy rendszerekben mutatkozik előnynek, nem pár száz soros fájlok írásakor.

Megjegyzendő még az elterjedt paradigmák közül a logika alapú programozás (ld. Prolog gyakorlatilag egyeduralkodóként), ami nagyon sok programozónak kifejezetten idegen, így fölöslegesen nagy nehézséget állít azok elé, akik egy egyszerű ellenőrzést kívánnak csak megvalósítani a programjukhoz. Egy bár funkcionális, de kevésbé szigorú (dinamikus) típusrendszerű nyelv az elgondolás szerint

nem rendelkezik ennyire kiemelkedően ezzel a problémával.

2.2. A nyelv családja és ismerősei

„Minden egy függvényként kerülhet definiálásra.” Ez a gondolat a LISP család tagjaként is tekinthetővé teszi ezt a nyelvet, és míg lehetőség van LISP jellegű kódot írni, nem ez az elsődleges cél. Magában a LISP másolásával pár nehézség lép fel, így a következőkben kifejtésre kerül, hogy helyette milyen módon kerül ez megvalósításra.

LISP család A LISP az egyik legöregebb programozási nyelvek közé tartozó funkcionális nyelv. Kiemelkedő jellegzetessége az ún. szimbolikus kifejezés (**symbolic expression** (*S-expression*); innentől **S-expr**) (eredetileg [McC60]), amelyik a kód elkülönülő kinézetét adja. Minden kifejezés egy (fv param1 param2...) formátumban áll elő, pl. (+ 1 1). Ez lehetővé teszi a nagyon egyszerű feldolgozást, hiszen a zárójelek és **prefix kifejezés** formájában megadott függvényhívás egyértelművé teszik a szöveg jelentését komolyabb számítások nélkül is (ellenben pl. egy **infix kifejezés**, amit a matematikából megszoktunk $1 + 1$).

A LISP nyelvek az egyik legdinamikusabb szintaktikát tudják biztosítani, ami a 3. tulajdonság szempontjából nagyon kedvező. Egy probléma adódik, miszerint a LISP nyelvek nem számítanak annyira elterjedtnek, mint a C vagy C++ nyelvek, így sok C és C++ programozó van, aki nem tud és valószínűleg nem is akar LISP kódot írni, ha ezt meg lehet oldani. Ez szembe megy a 4. tulajdonságban leírtakkal, így nem lenne ideális választás egy LISP szintaktikai szintű megvalósítása.

Egy kompromisszumot kell megállapítani a LISP általánossága és a megszokott C jellegű szintaktika között. Hasonlóan, mint a Tcl nyelv amelyik LISP jellegű szemantikával dolgozik egy C jellegű szintaktikát felhasználva. A Tcl azonban erősen procedurális nyelv, egy kis OO támogatással, amelyiket az évtizedek alatt összeszedett, így nem illeszkedik a funkcionális gondolatmenetre. Ebben a nyelvben a „minden procedúra” gondolatmenet mutatkozik meg, így még az egyszerű elágazás (**if**) is újradefiniálható a megfelelő procedúra definiálásra használt szintaktikával. Ennek a gondolatmenetnek a felhasználásával mutatom be a következő szakaszban az itt definiált nyelvet.

2.3. A nyelv felépítése

Nincs külön belépési függvény, a kód, ahogy megtalálható a fájlban fut. Ez hasonlít sok másik interpreterrel futtatott nyelvre.

Az előző rész elején említetteknek megfelelően az következne, hogy minden egy függvény. Egy kivétel azonban van, amivel a Tcl jobban megközelítette az általánosságot: a **let** nyelvi elemként kerül megvalósításra, ellenben a Tcl **proccal**. Ez a feldolgozás egyszerűsítését hivatott előidézni, mert így a **let** kódját annak feldolgozása közben el tudjuk különíteni más **tokenek**étől, ami lehetővé teszi, hogy egyáltalán értelmezni tudjuk a nyelvet.

2.3.1. Literálisok

Egyszerű literálisok a sok nyelvben megszokott szintaktikával létrehozhatóak:

Egészek 42, 1234

Tizedes törtek 4.2, 0.0, nem lehet leahagyni az egyik részét sem a számnak, tehát .2, 2. nem érvényesek.

Szöveg "szöveg", ez lehet több soros is.

Az utolsó literális fajta a blokk, amelyről a 2.3.4. szekcióban lesz szó.

2.3.2. Változók

Egy programozási nyelv bevezetésekor sok esetben a változókkal lehet kezdeni, mert elég egyszerűek azoknak is akik még nem programoztak, akik meg már tudnak programozni rögtön tudják hova kötni a már ismert nyelveik közé. Jelen esetben azonban fel kell ismerni valamit. A változók is igazából függvények. Mivel immutábilis értékekről van szó (leszámolva pár nem **tiszta (pure)** függvényt, amelyik be van építve IO műveletekre), így minden változó értelmezhető egy nulla aritású (nulláris) függvényként, amelyik mindig a definiálásakor kapott értéket adja vissza. Ennek felismerésével nem kell külön megvalósítani a változókat, elég tisztán a függvényeket általánosan használhatóvá tenni nulla paraméter estén is.

Megemlítenéd, hogy ez a szemantika eltér a LISP-es vagy Lambda kalkulusos értelmezéstől, amelyekben a nulla paraméteres függvények nem értelmezettek. Ez utóbbi esetében egyértelműen látszik, hiszen minden λ -absztrakció pontosan egy paramétert vár. Előbbi esetben kicsit bonyolultabb, de összességében egyszerű a levezetés: egy függvényhívás szintaktikája nulla bemenő paraméterrel a következő: (f) . Ez azonban $(f) \equiv (f.NIL) \equiv (f.())$, ami tehát azt jelenti, hogy f itt

```

let var 1
let var1/0 var
let fn/1 { |x|
  print "fn"
  var + x
}
let fn2/2 { |a b| (fn a) - b }

```

2.1. ábra. Példa változók és függvények definiálására

egy speciális egyedi értéket kap paraméternek, amelyikből csak pontosan egy van. Egyes nyelvekben ezt „Unit”-nak hívják.

2.3.3. Függvények

Minden függvényt a **let** kulcsszóval lehetséges bevezetni. Egy függvényhez szükséges megadni a nevét, aritását és a hozzá tartozó törzset. Ebből az első két tulajdonságot a Prolog jellegű funktor jelöléssel lehetséges megadni: egy név egy per-jellel elválasztva egy nemnegatív egész számtól, ami az aritás. Ezt hívjuk egy *szimbólumnak*, és a perjel egyik oldalán sem lehetséges szóközt hagyni. Ha az aritás nulla (egy változót definiál a **let** kifejezés), akkor a /0 elhagyható. Példa függvények definiálásra látható a 2.1. listában.

A függvény törzsének pontosan annyi szabad paramétertől kell függenie, amekora az aritása a függvény szimbólumának.

2.3.4. Blokkok

Egy függvény törzsében legtöbbször szükséges megadni egyéb másik függvényhívásokat, hogy a működést a kívánt módon meghatározzuk. Ehhez sokszor több, mint egy másik függvény hívására van szükség.

Egyes függvénytörzsek megadásához szükség van bevezetni ideiglenes, csak a függvényen belül értelmezett változókat (nulláris függvényeket), amelyek a szabad változóként működnek. Informatikai nomenklatúrában a függvények paramétereit meg kell tudni nevezni.

Mind kettő problémára a blokkok adnak megoldást: ezek olyan kapcsos zárójelekkel ({}), körbefogott függvényhívások sorozatai, amelynek az értéke az utoljára kiértékelt függvényhívás. Ilyen értelemben ezek is függvényként viselkednek, hiszen van valahány bemenetük és garantáltan adnak vissza egy másik értéket. Ebből a tulajdonságból már következik a több függvény hívásából adó problémára a megoldás. Szabad változókat pedig a Ruby szintaxisához hasonló módon a blokk legelején pipe karakterekkel (|) behatárolva lehetséges megadni, azonban szóközzel elválasztva. Mind kettő használatra látható példa a 2.1. listában.

```
let (+>)/2 left 2 { |a b| ... }  
let (?) /1 { |x| ... }  
1 +> 2  
?1
```

2.2. ábra. Példa operátorok definiálására és használatára

Az üres blokk Az üres blokk egy speciális egység, amelyik más nyelvek `null`, `nil` vagy `()` elemeinek feladatait látja el. Azonban ezektől a megvalósításoktól egy különbsége van: egy teljes értékű blokk. Meglepetést nem okozva, nulla paramétere van, de az előzőek alapján minden esetben vissza kell adnia egy értéket, mert a blokkok függvényként viselkednek. Így visszaadja saját magát, az üres blokkot.

Operátorok

Az operátorok olyan speciális függvények, amelyek csak szimbólum karakterekből állnak és vagy **prefix kifejezés**, vagy **infix kifejezés** formájában jelennek meg a kódban. Ennek megfelelően vonatkozik rájuk egy megkötés: csak egy vagy kettő paraméterük lehet. Az, hogy melyik, az a pre-, vagy infix tulajdonságukat határozza meg.

Az **infix kifejezés** formájú operátorok esetében van még két tulajdonság, amit figyelembe kell venni: a **precedencia** (*precedence*) és **asszociativitás** (*associativity*)¹. Asszociativitásból lehet **left** vagy **right** a bal és jobb asszociativitásért, míg precedencia egy 0-tól 10-ig tartó egész érték bármelyike lehet. Minél magasabb a szám, annál hamarabb kerül kiértékelésre.

Mivel az operátorok is függvények, így definiálni a **let** kulcsszóval lehetséges: az operátor zárójelek között aritásával megadva, majd az asszociativitás és precedencia tulajdonsága, végül a megszokott törzs. Példa található a 2.2. listában.

Logikai kifejezések

Feltűnő lehet a **true** és **false** kulcsszavak/literálisok hiánya. Ez azonban könnyen feloldható: az üres blokk, mint ahogy az előzőekben bevezetésre került számít „hamis” értéknek, minden más „igaz”. Bár ezt a nyelv nem kényszeríti ki, a beépített logikai értékeket használó függvények ezt a konvenciót használják.

Könnyedségként azonban, két nulláris függvény is definiált: **true/0** és **false/0**. Ezek a nevüknek megfelelő logikai értéket adnak vissza, így a kódban jobban átlátható lesz, mint üres blokkok (hamis) vagy fura konstansok (igaz) használatával.

¹Nem a matematikai értelemben kell itt értelmezni, hanem a nyelvek definiálásánál használt értelmezésben. ld. szójegyzék a definícióért.

```

1  -- nagyon nem idiomatikus Haskell kód,
   lustaság példa
2  bar = do
3      putStrLn "bar"
4      return 42
5
6  foo x = do
7      putStrLn "foo"
8      x
9
10 main = do
11     foo bar
12     putStrLn "main"

```

(a) Lusta Haskell kód

```

1  let bar {
2      putStrLn "bar"
3      42
4  }
5  let foo/1 { |x|
6      putStrLn "foo"
7      x
8  }
9  foo bar
10 putStrLn "main"

```

(b) Lusta C4 kód

```

foo
bar
main

```

(c) Kimenet

2.3. ábra. Lustaság példa

2.3.5. Lustaság

Egyes (funkcionális) nyelvek ún. **lusta** (*lazy*) kiértékelést alkalmaznak. Ilyen például a Haskell. Ez azt jelenti, hogy egy függvényhívás, változó kötés stb. addig ténylegesen nem történik meg, amíg a program olyan lépést nem hajt végre, amelyik szükségessé teszi annak a kiszámítását. Példa erre a viselkedésre látható a 2.3a. listázásban.

Mivel minden függvény, így az if és egyéb nyelvi szerkezetek is. Emiatt szükség van annak a lehetőségére, hogy úgy vegyen át egy-egy függvény paramétert, hogy az ne értékelődjön ki. Az előbb említett if erre egy jó példa: ha az első paraméterre (logikai feltétel) igaz akkor a harmadik paraméterét nem kell/szabad kiértékelni. Analóg módon a hamis esetben a második paraméterrel. Ennek megoldására jó lehetőséget nyújt az általános **lusta** kiértékelés minden paraméterben. Látható ennek eredménye egy, a Haskell kóddal analóg példában a 2.3b. listázásban.

3. fejezet

Megvalósítás

Ebben a fejezetben részletezésre kerül az előző fejezetben ismertetett programozási nyelv megvalósítása. Elsőként a szöveg értelmezésére térek ki, majd a bináris generálásának folyamatára.

3.1. A lexikális feldolgozás

A lexikai feldolgozás saját megvalósítás. A PCRE2 könyvtárat [Phi15] és annak a megfelelő JIT támogatását használja, hogy a lehető leggyorsabban tokenekre tudjuk bontani a szöveget.¹ Minden token egy saját típusként jelenik meg, a lexikai elemző pedig ezeknek egy `std::variant` objektumba csomagolt értékét generálja.

Egy fontos részlet, hogy a mind a lexikai elemző, mind az értelmező elkerüli a memória foglalatást ahol csak lehetséges, hogy ezzel is hatékonyabb legyen az értelmezés folyamata. Ehhez egy fájl feldolgozása a **memóriába-lapozás (memory mapping)** művelettel kezdődik. Innentől minden token, és minden AST csúcspont, amelyik a forrásfájl szövegére hivatkozik, csak pointert tárol erre a nagy memóriára, ezzel elkerülve az egyes (rövid) szövegdaraboknak való külön memória foglalatást.

3.2. Az értelmező

Az értelmező nagy részben megszokott módszereket alkalmaz. Egy manuálisan megvalósított, legtöbb részében szokásos LL(1) algoritmus szolgál az alapjául. Az érdekesnek vélt részeit az alábbi alszekciók mutatják be.

¹Jelenleg a konstans szövegeket is reguláris kifejezéssé alakítva kezeli a c4c. Ez nem optimális, de a megvalósításban megvan a hely egy megfelelő szöveg illesztőnek.

```

1  # test.c4
2  let bar {
3      println "bar"
4      42
5  }
6  let foo/1 { |x|
7      println "foo"
8      x
9  }
10 foo baar
11 println "main"

```

(a) Hibás kód

```

... \test.c4:10:21: error: unknown symbol referenced in function call:
      baar
    9 |           foo baar
      |           ^~~~
fatal: parser failure: bad token

```

(b) Kimenet

Megjegyzendő, hogy az „error:” és „baar” szövegrészeket piros színűek, hogy jobban láthatóak legyenek hosszabb kimenetben.

3.1. ábra. A c4c hibaformátuma

3.2.1. Hibaüzenetek

Az egyik legfontosabb tulajdonsága egy fordítónak felhasználói szempontból, a helyesség mellett, hogy a generált hibaüzenetek érthetőek és jól olvashatóak legyenek. Ezzel egyet fog érteni mindenki, aki például $\text{T}_{\text{E}}\text{X}$ kimenetet olvasott. Ennek megfelelően a C4 értelmező próbál a lehető legjobb formátumú és legtöbb információval ellátott jelölést használni a kimenetére. Ehhez egy jól ismert C++ fordító, a **GNU Compiler Collection (GCC)** formátumát használja. Amellett, hogy relatíve jól olvasható formátum, előnye a 4. tulajdonság segítségével is jelentős, hiszen C vagy C++ fejlesztők vélhetőleg ismerősek lesznek ezzel a formátummal. A hibás bemenetre adott válaszra példa látható 3.1. ábrán.

3.2.2. Infix operátorok

A C4 nyelv támogat saját operátorokat. Ez jelentősen megnehezíti az értelmezést, mert az operátorok a program értelmezése közben kerülnek létrehozásra saját **asszociativitás** és **precedencia** tulajdonságokkal, amelyek a program értelmezését lényegesen befolyásolják.

Mivel egy általános recursive–descent értelmezőt valósít meg a c4c, ezért statikus operátorok esetén az ún. „precedence climbing” avagy precedencia mászássos algoritmust valósítják meg sok esetben, pusztán egyszerűsége miatt. Ez a különböző precedenciájú operátorokat a nyelvtan különböző szabályaiba osztja, és ettől az egyszerű rekurzív értelmező automatikusan megfelelő sorrendben fogja őket beépíteni a készített **AST** rész fába.

A probléma azonban, hogy a C4 operátorok dinamikusan kerülnek definiálásra, így a nyelvtan szabályai változnak menet közben. Emiatt szükséges lenne fenntartani annyi listát, amennyi különböző precedencia szint van (10) és azoknak megfelelően végrehajtani valamiféle dinamicitással az egyes lépésekben a rekurziót. Ez, bár működik, minden kifejezést nagyon mély rekurzió árán kiszámít-

hatóvá tesz: egy egyszerű 1 érték, 10 rekurzív függvényhívást fog igénybe venni, ami nő, ha több a precedencia szintek száma. Egy összetettebb kifejezésben ez jóval több tud lenni, mert minden operátornál újra és újra belépünk egy rekurzív hívásba az adott szintről.

Ezt elkerülendő, egy hatékonyabb algoritmust mutat be a BCPL nyelv és fordítójának eredeti leírása. [RWS84] Ebben (BCPL nyelven és szövegesen) került leírásra az az algoritmus, amit a c4c is használ. (Modernebb megvalósítás és leírás is létezik: pl. ANTLR3-hoz [Sam08]) Röviden összefoglalva: addig ciklikusan ismételjük az adott rekurzió szinten lévő beolvasást, ameddig vagy jobb–asszociatív vagy nem azonos precedencia szinten lévő operátort találunk, és csak ezekben az esetekben kezdeményezünk új rekurziót operátor értelmezésre. Látható, hogy ennek előnye, hogy egy részét a rekurzív hívásoknak egy ciklussá alakítja.

A dinamikus operátorok értelmezéséhez annyi módosítást kell eszközölni, hogy az egyes operátorokat nem egy statikus elágazás, hanem egy dinamikus táblázat szerint értelmezzük.

3.2.3. Függvényhívások

Mivel a LISP jellegű zárójelezést elhagyjuk, így sok esetben magától adódóan nem egyértelmű a nyelvtan. Igazából kontextus függetlennek sem mondható, a megoldás miatt, amelyik lehetővé teszi a zárójelek elhagyását. Ezt részletezi az alábbi bekezdés.

Minden függvényt a `let` kulcsszóval lehet bevezetni, így feldolgozás közben egyértelművé válik, hogy milyen függvények léteznek a kódban, és azoknak mekkora az aritása. Ezt fel lehet használni két dologra: egyértelműsíthető az, hogy melyik függvényhez melyik paraméter tartozik. Mivel mindig ismert a szükséges paraméterek száma, így egyértelműen megadható egy–egy kifejezés helyes zárójelezése, amelyik azonos lesz azzal, amit akkor kaptunk volna, ha LISP jelleggel minden zárójelet kiírtunk volna.

Emellett lehetséges már értelmezés közben a teljes névfeloldást végrehajtani. Ennek következtében az összes olyan függvényt, amit a kódban definiálunk egyértelműen tudjuk hivatkozni extra számítások nélkül.

Ez teszi lehetővé még azt is, hogy a nyelvi elemnek tűnő függvények úgy tudjanak működni, ahogy az elvárás egy nyelvi elemtől. Mivel az `if/3` egy függvény, így ugyan úgy lehet formázni, mint bármelyik másik függvényt. Bármelyik függvényt azonban úgy lehet formázni, ahogy kívánja a fejlesztő, így mindenki a számára kedvező módon definiálhatja a projekt formázási stílusát.

3.3. LLVM IR generálása

A fordító/értelmező hatékonyságát megoldandó a generált kód felhasználja az LLVM projekt által feljesztett, nyílt forráskódú LLVM fordító backendet. Itt történik meg a kód optimalizációja, illetve a megfelelő hardverre való binárisba fordítása.

Féléves munka korlátja A félév során csak egy szokásos fordító (compiler) készítésére volt elegendő idő (c4c), de a terv az LLVM által támogatott **JIT** rendszer használata. Utóbbi azonos formában várja a bemenetet, mint a klasszikus fordítást végrehajtó része az LLVM-nek, így elsőként a fordítóval való megvalósítás könnyedségét kihasználva elkészült a fordító, és a kapott **Internal Representation (IR)** kimenetet generáló kód majd átemelhető lesz a **JIT** rendszerbe.

Az értelmezésből kapott **AST**-t egy aciklikus visitor mintát felhasználva lehetséges bejárni. A bejárás közben a megfelelő

3.3.1. Változók

Bár a C4 nyelv absztrakt leírásában elkerülhetőek a változók azzal, hogy nulla paraméterű függvényként kerülnek definiálásra, alacsony, már-már gépi kód, szinten ez a megközelítés pár nehézségbe ütközik. Legnagyobb probléma a **lusta** kiértékelés szemantikájából következik. A processzor szinten egy függvény meghívása drága, a különböző stack kezelés és hozzá tartozó műveletek miatt. Ezzel szemben egy lokális változó jelentősen olcsóbb, mondhatni ingyen van, mert a stack terület, amire szüksége van az a függvény létrehozásánál magától létrejön, értékadásnál csak az értékadó műveletet kell kifizetni. Ezt kihasználva két részre lehet bontani a változók kezelését:

Elsőként a teljesen konstans értékű változók, például, amelyek egy-egy (fix méretű) literális értékét tárolják. Ezeknek ténylegesen létrehozni egy függvényt, amelyik visszaadja a konstans értéket fölösleges. Bár az LLVM optimalizáló kifejezetten nagy hatékonysággal tudja/tudná ezeket a „fölsleges” függvényeket beágyazni a hívási helyeikre, nem cél direkt lassítani az optimalizálás folyamatát az aktívan szubotimális kód generálásval, amelyik még a szemantikában sem változtat semmit. Mivel egy-egy konstans érték kiszámítása nem okoz semmiféle mellékhatást, ami alapján eldönthető lenne, hogy **szorgos (eager)** kiértékelés történt-e, így ezt az optimalizációt generálás közben végrehajtja a c4c.

Olyan változók, azonban, amelyek valamilyen kifejezés eredményeként születnek csak a teljes számítás ellenőrzésekor állíthatóak, hogy mellékhatás mente-

se, amellet, hogy számításuk ismeretlen ideig is tarthat. Ezért nem lehet/szabad, csak úgy előre kiemelni ezeknek az értékeknek a számítását, hogy a helyes **lusta** szemantikát fenntartsuk. Megjegyzendő, hogy általánosan ezt az optimalizálást lehetséges megcsinálni. Például a GHC Haskell fordító fejlesztői több éve dolgoznak ezen és hasonló problémákon optimalizációs céllal [SJV14] [SJV17].

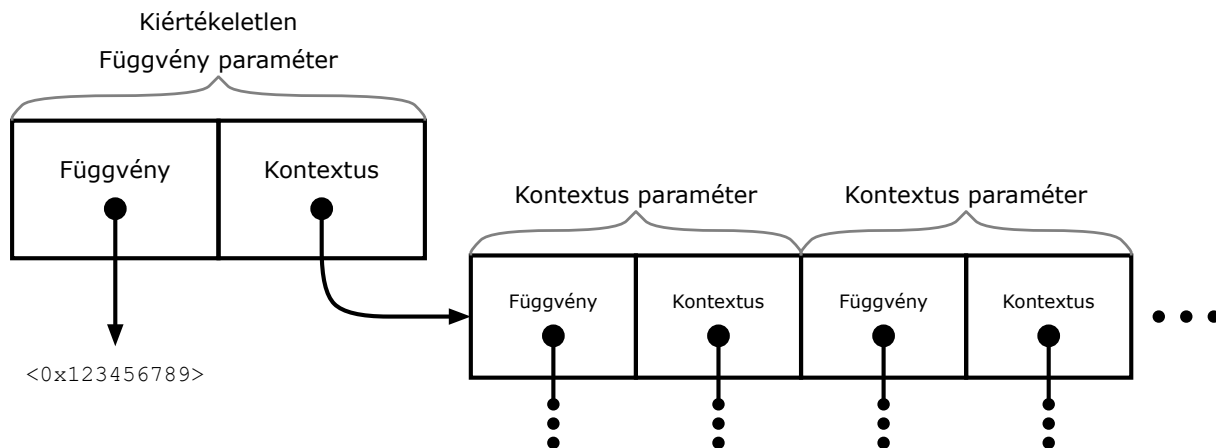
Viszont a c4c fordítónak nincs erre olyan szinten igénye, hogy a Haskell fordító leírásában a Haskell **AST**-ben hajtják végre, c4c azonban az elsődleges optimalizációját az LLVM rendszerre bízta. Utóbbi pedig LLVM **IR** szinten tudja, és hajtja is végre az utasítás átrendezést, így sokkal primitívebb elemekből tudja megvalósítani ugyan azt, általánosan. Ez teszi azt is lehetővé, hogy függvényként kezeljük a megfelelő nyelvi elemeket is. Mivel általános esetben működik a primitív LLVM **IR** instrukciók beágyazása, átrendezése és egyéb optimalizációs művelet, így ugyan az a mechanizmus ami lehetővé teszi számunkra a változók kiértékelésével való egyszerűsítést, az **if/3** és egyéb függvények általános kezelését is nyújtja.

3.3.2. Lusta paraméterek

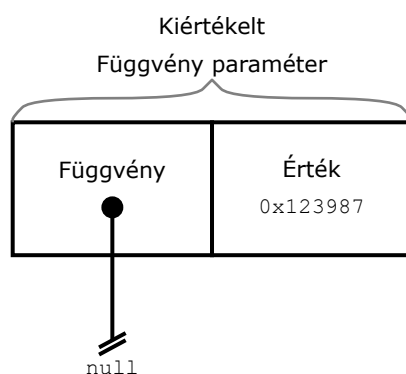
Az LLVM **IR**, mint a legtöbb nyelv **szorgos** kiértékeléssel dolgozik: egy függvény paramétereit ki kell értékelni, mielőtt a függvény meghívásra kerül. A C4 nyelvben azonban egy Haskell-szerű **lusta** paraméterkiértékelésre van szükség. Ehhez egy speciális paraméterátadásra van szükség: Minden paraméter helyett egy-egy pointer-t adunk át egy-egy két elemű pointerekből álló tömbbe.² A tömb első eleme egy pointer arra a már **IR** szintű, generált függvényre, amelyik az adott értéket számítja ki. Ha ez nem direktben egy függvény, akkor egy anonim függvény keletkezik, amelyik **closure**-ként viselkedik. (ld. alább) A második érték, pedig az a kontextusra mutató pointer, amelyik az esetleges **closure**-nek szükséges, amelyikre a pointer az első mezőben van. Amennyiben nem **closure** az átadott függvény, akkor ez az érték **null**. A 3.2a. ábrán látható egy-egy koncepcionális ábra a paraméterek memóriabeli elhelyezkedéséről.

Kiértékelés során csak egyszer kívánatos egy-egy paramétert kiszámolni, hogy a szemantikának megfelelő legyen a lefutás. Ehhez a kiértékelés után a következő lépéseket hajtja végre a hívott függvény (amelyik paramétere éppen kiértékelés alatt van). A kiértékelésből kapott értékekkel felülírja a második értéket a tömbben, majd, hogy a következő hívások esetén ismert legyen a paraméter kiértékelte volta, a függvény mutatóját **null**-ra állítja. A 3.2b. ábra ezt mutatja az állapotot be. Hogy ezt felismerje, minden hivatkozás elsőként az első paraméter nem **null** értékét ellenőrizni, és aszerint vagy visszaadja az második értéket, vagy ha nem

²Itt feltételezzük, hogy pointer mérete az 64 bit, a megvalósítás ennél trükkösebben jár el, és működik 32 bit méretű pointerekkel is.



(a) Kiértékeletlen függvény paraméter



(b) Kiértékelt függvény paraméter

3.2. ábra. Paraméterek memóriabeli ábrázolása

`null`, akkor a megfelelő módon kiértékeli.

3.3.3. Closure blokkok

Egy blokk definiálásakor hivatkozhat egy olyan függvényre/változóra, amelyik a rajta kívül eső blokkban lett definiálva, de nem globális. (Belátható, hogy csak globálisokra hivatkozáskor nincs semmi probléma.) Ilyenkor a számítás során szükség van arra az értékre, amelyik a hívás pillanatában érvényes. Az ilyen extra feltételekkel rendelkező blokkok az ún. **closure** blokkok.

A megvalósításhoz a C4 szinten adott aritásnál egyel több paraméterük van: egy extra mutató paramétert kapnak a legelső pozícióban. Ebben ahány függvény fölött számítanak **closure**nek, annyiszor két mező található. Ezek az értékek inentől megegyeznek a paraméterek átadásánál írtakkal.

Rövidítésjegyzék

AST Abstract Syntax Tree. 4, 11, 12, 14, 15

DSL Domain-Specific Language. 4

GCC GNU Compiler Colletion. 12

IR Interl Representation. 14, 15

JIT Just-In-Time compiler. 3, 11, 14

Szójegyzék

asszociativitás Két azonos operátor egymásutánjakor a zárójelezés irányát meghatározó tulajdonsága egy operátornak. Lehet jobb vagy bal. 9, 12

closure Olyan blokk, amelyik saját számításaihoz egy rajta kívül definiált, de nem globális függvényt használ.. 15, 16

infix kifejezés Olyan kifejezés, amelyikben az elvégzendő művelet a kifejezés közepén található. pl. a legtöbb szokásos matematikai művelet jelölés, $1 + 2$. 6, 9, lásd **prefix kifejezés**

lusta Olyan kiértékelés, amelyikben egy kifejezés csak akkor értékelődik ki teljesen, ha van olyan másik kifejezés, amelyik kiértékelődik, és használja az első kifejezés eredményét.. 1, 10, 14, 15

memóriába-lapozás Egy folyamat, amelyen keresztül az operációs rendszert felhasználva egy fájl tartalmát memóriában elérhetővé alakítjuk.. 11

precedencia Két operátor egy kifejezésben használatakor, azok kiértékelési sorrendjét megadó sorrendezése minden operátornak. 9, 12

prefix kifejezés Olyan kifejezés, amelyikben az elvégzendő művelet a kifejezés legelején található. pl. egy LISP **S-expr**, $(+ \ 1 \ 2)$. 6, 9, lásd **infix kifejezés** & **S-expr**

S-expr Symbolic expression. Olyan egymásba ágyazott listákat leíró szintaktika, amelyet többek között a LISP nyelvcsalád használ, mind kód, mind adat tárolására. 6, 18

szorgos Olyan kiértékelés, amelyikben az összes kifejezés azonnal kiértékelésre kerül, attól függetlenül, hogy használja-e annak a végeredményét másik kifejezés. 14, 15

tiszta Olyan függvény, amelyik megfelel a matematikai függvény definíciójának: adott bemenetekre mindig ugyan azt az eredményt adja és nincs semmiféle mellékhatása. Tehát olyan megfigyelhető változást nem eredményez, amelyik nem a kimeneti értéke. [7](#)

token Elemi egységei egy–egy nyelvtannak, amelyek a szöveg egy folyamatos részéhez rendelnek szemantikus értelmet. Tartalmazhat valamilyen formában értelmezett részeket is, pl. egy szám értéke már nem csak szöveg, hanem szám formában is.. [7](#), [11](#)

Irodalomjegyzék

- [LLV] LLVM Project, „Getting Involved — Extra Clang Tools 21.0.0git documentation.” [Online]. Available: <https://clang.llvm.org/extra/clang-tidy/Contributing.html#writing-a-clang-tidy-check>
- [McC60] J. McCarthy, „Recursive functions of symbolic expressions and their computation by machine, Part I,” *Commun. ACM*, vol. 3, no. 4, pp. 184–195, Apr. 1960. [Online]. Available: <https://dl.acm.org/doi/10.1145/367177.367199>
- [Phi15] Philip Hazel, „PCRE - Perl Compatible Regular Expressions,” Jan. 2015. [Online]. Available: <https://www.pcre.org/>
- [Rub] RuboCop Project, „Development :: RuboCop Docs.” [Online]. Available: <https://docs.rubocop.org/rubocop/development.html>
- [RWS84] M. Richards and C. Whitby-Strevens, *Basic Combined Programming Language, the language and its compiler*, reprint ed. Cambridge: Univ. Pr, 1984.
- [Sam08] Sam Harwell, „Operator precedence parser - ANTLR 3 - Confluence,” Aug. 2008. [Online]. Available: <https://theantlruguy.atlassian.net/wiki/spaces/ANTLR3/pages/2687077/Operator+precedence+parser>
- [SVJ14] I. Sergey, S. P. Jones, and D. Vytiniotis, „Modular, higher-order cardinality analysis in theory and practice | Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages,” Jan. 2014. [Online]. Available: <https://dl.acm.org/doi/10.1145/2535838.2535861>
- [SVJ17] —, „Theory and Practice of Demand Analysis in Haskell,” *DRAFT*, Mar. 2017. [Online]. Available: <https://www.microsoft.com/en-us/research/wp-content/uploads/2017/03/demand-jfp-draft.pdf>